CyberSOC Platform

Production Deployment Playbook

Step-by-Step Cutover Procedure for General Availability
Djezzy Algeria | National SOC Operations Center

Document Version:	2.0.0-GA
Classification:	Internal - Operational
Effective Date:	2026-08-26
Review Cycle:	Quarterly or After Major Changes
Owner:	CyberSOC Platform Team

Table of Contents

1. Executive Summary

This Deployment Playbook provides a comprehensive, step-by-step guide for executing the production cutover of the CyberSOC Platform to General Availability (GA) status for Djezzy Algeria's National Security Operations Center. The document outlines all critical procedures, validation checkpoints, rollback mechanisms, and escalation paths necessary to ensure a successful deployment with minimal service disruption and zero data loss.

1.1 Scope and Objectives

The primary objective of this deployment is to transition the CyberSOC Platform from its current staging environment to a fully operational production state capable of handling enterprise-grade security operations at scale. The platform must achieve sustained throughput of 10,000 events per second (EPS) while maintaining sub-second response times for critical alert triage workflows. Additionally, the deployment must ensure compliance with ANRT (Autorite de Regulation de la Poste et des Communications Electroniques) regulatory requirements, including mandatory audit logging with 7-year retention periods and real-time threat intelligence sharing capabilities.

1.2 Key Success Criteria

Criterion	Target Value	Measurement Method
System Availability	99.95% uptime	Prometheus / Grafana monitoring
Event Processing Rate	>10,000 EPS sustained	Kafka consumer lag metrics
API Response Time (P95)	<500ms	Application performance monitoring
Database Query Performance	<100ms P95	PostgreSQL query statistics
Security Posture Score	>95%	Automated security scanning
Data Loss Tolerance	ZERO data loss	Transaction log verification
Recovery Time Objective (RTO)	<15 minutes	DR drill execution time
Recovery Point Objective (RPO)	<1 hour	Backup replication lag

1.3 Deployment Timeline Overview

The complete deployment timeline spans approximately two weeks, structured into six distinct phases with built-in validation gates between each phase. Phase 1 begins at T-7 days (seven days before target go-live date) with infrastructure preparation activities including Kubernetes cluster provisioning, network policy configuration, and storage class setup. Phase 2 covers database migration from SQLite development databases to PostgreSQL HA clusters with read replica configuration and connection pooling via PgBouncer. Phase 3 represents the actual application deployment on T-Day, involving container image promotion, rolling updates across all

microservices, and ingress controller configuration. Phase 4 focuses on comprehensive validation testing including integration tests, load testing at target EPS rates, and security penetration testing. Phase 5 executes the actual traffic cutover from staging to production endpoints. Finally, Phase 6 establishes the hypercare support period with enhanced monitoring, rapid incident response, and stakeholder communication protocols.

2. Pre-Deployment Checklist

Before initiating any deployment activities, the deployment team must verify that all prerequisite conditions have been met and all stakeholders have provided necessary approvals. This section provides a comprehensive checklist organized by category to ensure nothing is overlooked during the high-pressure deployment window. Each item must be explicitly checked off by the responsible party with timestamp and signature documentation retained for audit purposes.

2.1 Infrastructure Readiness

Item	Status	Owner	Verified
Kubernetes cluster (production) provisioned and accessible	[ ]	Platform Team	
All node pools configured with correct labels and taints	[ ]	Platform Team	
Storage classes created (premium-ssd, premium-nvme)	[ ]	Storage Team	
Network policies reviewed and approved by security	[ ]	Security Team	
TLS certificates provisioned via cert-manager	[ ]	Security Team	
External DNS records configured for production domains	[ ]	Network Team	
Load balancer IPs reserved and configured	[ ]	Network Team	
Monitoring stack (Prometheus/Grafana) deployed	[ ]	Observability	
Alertmanager routes configured with PagerDuty/Slack	[ ]	Observability	

2.2 Database Readiness

Item	Status	Owner	Verified
PostgreSQL HA cluster (3 primary + 2 replicas) running	[ ]	DBA Team	
PgBouncer connection pooling configured and tested	[ ]	DBA Team	
Database backups verified (full + WAL)	[ ]	DBA Team	
Migration scripts tested against staging clone	[ ]	DBA Team	
Connection strings documented in secrets manager	[ ]	Security Team	
Performance tuning parameters applied	[ ]	DBA Team	
Replication lag < 1 second confirmed	[ ]	DBA Team	
Row-Level Security policies enabled	[ ]	Security Team	

2.3 Application & Security Readiness

Item	Status	Owner	Verified
Container images built and pushed to registry (tagged v2.0.0-prod)	[ ]	DevOps	
Helm chart values-production.yaml reviewed	[ ]	DevOps	
.env.production template completed and validated	[ ]	Security Team	
JWT secrets generated and stored in Vault/K8s Secrets	[ ]	Security Team	
Rate limiting configuration tested	[ ]	Security Team	
CORS policies configured for production domains	[ ]	Security Team	
WAF rules imported and activated	[ ]	Security Team	
Audit logging enabled with correct retention	[ ]	Compliance	
All API endpoints passing security scan	[ ]	Security Team	
Penetration test completed with no critical findings	[ ]	Security Team	

3. Phase 1: Infrastructure Preparation (T-7 days)

Phase 1 establishes the foundational infrastructure required for the production deployment. This phase begins seven days before the target go-live date to provide adequate buffer time for addressing any unexpected issues that may arise during cluster provisioning or network configuration. All activities in this phase should be performed during business hours to ensure maximum availability of vendor support and internal subject matter experts.

3.1 Kubernetes Cluster Setup

The production Kubernetes cluster must be provisioned according to the specifications defined in the values-production.yaml Helm chart configuration. The cluster should consist of a minimum of nine worker nodes distributed across three availability zones for high availability. Node pools should be dedicated by workload type: critical workloads (API servers, SIEM processors) run on nodes with premium NVMe storage attached, while batch processing workloads (reporting, analytics) utilize standard SSD storage classes. Each node pool must have appropriate taints and tolerations configured to prevent resource contention between latency-sensitive and batch workloads.

Key Commands:

```
# Create production namespace
kubectl create namespace cybersoc

# Apply network policies (zero-trust model)
kubectl apply -f k8s/cert-manager/certificates.yaml

# Verify cluster nodes
kubectl get nodes -L node-type -L workload-class

# Check resource quotas
kubectl describe resourcequota -n cybersoc
```

3.2 Storage Configuration

Persistent storage must be provisioned using Kubernetes StorageClass resources mapped to your cloud provider's block storage offerings. For the CyberSOC Platform, three distinct storage classes are required: premium-nvme for hot Elasticsearch data and PostgreSQL WAL files requiring maximum IOPS; premium-ssd for warm data, application logs, and Redis persistence; and standard storage for backup repositories and cold archive data. Each StorageClass should be configured with appropriate retention policies, encryption-at-rest enabled using cloud provider KMS keys, and snapshot schedules aligned with the RPO requirements defined in the success criteria.

3.3 TLS Certificate Provisioning

TLS certificates for all production domains must be provisioned using cert-manager with the Let's Encrypt production CA (or an internal enterprise CA if operating in an air-gapped environment). The certificate resources defined in k8s/cert-manager/certificates.yaml should be applied to create Certificate resources for soc.djezzy.dz, api-soc.djezzy.dz, grafana.soc.djezzy.dz, and internal service mesh communication. Certificates should be configured with automatic renewal 15 days before expiration, and monitoring alerts should be configured to notify the

security team if any certificate approaches expiry within 30 days.

3.4 Phase 1 Validation Gate

Before proceeding to Phase 2, the following validation checks must pass successfully. Any failure requires immediate remediation and re-execution of the affected step before advancement can be approved. The deployment lead and platform architect must jointly sign off on Phase 1 completion.

Validation Check	Expected Result	Actual	Pass/Fail
Cluster health check	All nodes Ready		
DNS resolution for prod domains	Resolves to LB IP		
TLS certificate issuance	Certificates Ready		
StorageClass provisioning	3 SCs available		
Network policy enforcement	Default-deny active		
Resource quota limits	Quotas applied		

4. Phase 2: Database Migration (T-3 days)

Phase 2 executes the critical database migration from SQLite (development/staging) to PostgreSQL HA (production). This phase begins three days before go-live to allow sufficient time for data validation, performance tuning, and potential rollback if issues are discovered. The migration uses the execute-staging-migration.sh script which has been specifically designed for idempotent execution and includes comprehensive error handling and rollback capabilities at each step.

4.1 Pre-Migration Backup

Before initiating any migration activities, a complete backup of the existing SQLite database must be created and verified. The backup includes both the raw database file and a SQL dump for human-readable inspection. Additionally, all application state including sessions, cache entries, and queued jobs should be captured to enable exact point-in-time restoration if required. Backup integrity must be verified by performing a test restore to a temporary location and comparing row counts against the source database.

```
# Execute the staging migration script
chmod +x scripts/database/execute-staging-migration.sh
./scripts/database/execute-staging-migration.sh

# Verify backup creation
ls -lah backups/staging_migration_*/

# Check migration report
cat backups/staging_migration_*/migration_report.txt
```

4.2 Schema Migration

Schema migration converts the Prisma schema from SQLite provider to PostgreSQL provider, accounting for dialect-specific differences in data types, indexing strategies, and constraint definitions. The migration process generates a PostgreSQL-optimized schema file (schema-postgresql.prisma) and executes Prisma's db push command to synchronize the database structure. For production deployments, it is strongly recommended to use Prisma Migrate with versioned migration files rather than db push to maintain an auditable history of schema changes and enable deterministic rollbacks.

4.3 Data Migration & Verification

Data migration transfers existing records from SQLite tables to their PostgreSQL counterparts while preserving referential integrity and data fidelity. The migration script handles common type conversions automatically (INTEGER PRIMARY KEY to SERIAL, BLOB to BYTEA, DATETIME to TIMESTAMPTZ) but manual intervention may be required for complex data types or custom SQL functions. Post-migration verification compares row counts between source and destination tables, samples random records for content accuracy, and validates that all foreign key relationships remain intact.

Migration Verification Table Example:

Table Name	SQLite Rows	PostgreSQL Rows	Match	Notes

users	1,247	1,247	YES	
sessions	5,891	5,891	YES	
alerts	127,453	127,453	YES	
incidents	3,291	3,291	YES	
audit_logs	892,104	892,104	YES	Large table - verify sample

5. Phase 3: Application Deployment (T-Day)

Phase 3 represents the main deployment event where containerized applications are rolled out to the production Kubernetes cluster. This phase occurs on T-Day (the target go-live date) and follows a carefully choreographed sequence designed to minimize downtime and enable rapid rollback if any anomalies are detected. The deployment utilizes Kubernetes rolling update strategy with configurable surge and unavailable parameters to maintain service capacity throughout the update process.

5.1 Pre-Deployment Final Checks

In the hours immediately preceding the deployment window, conduct a final verification of all systems. Confirm that the production Helm release does not currently exist (or is in a safe state for upgrade). Verify that all container images tagged v2.0.0-prod are available in the registry and pullable by all worker nodes. Validate that external dependencies (PostgreSQL, Redis, Elasticsearch, Kafka) are reachable from the cluster network and responding within acceptable latency thresholds. Ensure that all team members are available via the designated communication channel (Slack bridge line or video conference).

5.2 Helm Deployment Execution

```
# Add/update Helm repository
helm repo add cybersoc https://charts.soc.djezzy.dz
helm repo update

# Deploy with production values
helm install soc-platform ./k8s/helm/soc-platform \
  -f ./k8s/helm/soc-platform/values-production.yaml \
  --namespace cybersoc \
  --create-namespace \
  --wait --timeout 600s

# Monitor rollout status
helm status soc-platform -n cybersoc
kubectl rollout status deployment/soc-platform-api -n cybersoc

# Check pod health
kubectl get pods -n cybersoc -l app.kubernetes.io/name=soc-platform
```

5.3 Health Check Validation

Following the Helm chart installation, validate that all pods have reached the Ready state and are passing liveness and readiness probes. Execute the application-level health check endpoint to verify database connectivity, cache availability, and inter-service communication. Confirm that metrics are being exported to Prometheus and that no error alerts are firing in Alertmanager. Document the exact timestamps when each component entered the Ready state for post-deployment analysis.

Component	Health Endpoint	Expected Response	Status
API Server	/api/health/live	{"status":"ok"}	

Readiness Probe	/api/health/ready	{"db":"ok","redis":"ok"}	
PostgreSQL	port 5432 TCP	Connection accepted	
Redis	port 6379 TCP	PONG response	
Elasticsearch	port 9200 HTTP	cluster_name returned	
Kafka	port 9092 TCP	Metadata response	

6. Phase 4: Validation & Testing (T+1 day)

Phase 4 focuses on comprehensive validation of the deployed system to confirm that all components are functioning correctly under realistic load conditions. This phase begins on T+1 (one day after initial deployment) to allow the system to stabilize and for initial telemetry baselines to be established. Testing activities progress from basic smoke tests through integration validation to full-scale load testing and security assessment.

6.1 Smoke Testing

Smoke tests verify basic functionality across all major user journeys without requiring extensive test data setup. These tests should complete within 30 minutes and provide immediate confidence that the core application is operational. Test scenarios include user authentication (including MFA flow), dashboard loading with real-time data feeds, alert creation and acknowledgment workflow, incident creation from an alert, and report generation export functionality. Any smoke test failure blocks progression to deeper testing levels until root cause is identified and remediated.

6.2 Integration Testing

Integration testing validates end-to-end workflows that span multiple system components and external integrations. Key test scenarios include SS7 message ingestion and fraud rule evaluation pipeline, threat intelligence feed synchronization (MISP, OpenCTI), SOAR playbook execution with external ticketing system integration, ANRT compliance report generation and submission, and real-time SSE stream delivery to connected dashboard clients. Each integration test should validate both the happy path and common error conditions to ensure proper error handling and graceful degradation.

6.3 Load Testing

Load testing validates that the system can sustain the target throughput of 10,000 events per second while maintaining acceptable response times. Use the k6 load testing scripts located in performance/load-testing/ to simulate realistic traffic patterns including burst scenarios (2x baseline for 5 minutes) and sustained peak load (1.5x baseline for 1 hour). Monitor key metrics including API response time percentiles (P50, P95, P99), error rates, database connection pool utilization, Kafka consumer lag, and Elasticsearch indexing latency. Document baseline performance metrics for ongoing comparison and capacity planning.

```
# Run load test targeting 10K EPS
k6 run performance/load-testing/k6-10k-eps-target.js \
  --env BASE_URL=https://soc.djezky.dz \
  --env TARGET_EPS=10000 \
  --duration 1h

# Run dashboard load test
k6 run performance/load-testing/k6-dashboard-load.js \
  --env BASE_URL=https://soc.djezky.dz \
  --vus 100 --duration 30m
```

7. Phase 5: Cutover & Go-Live (T+2 days)

Phase 5 executes the actual traffic cutover from the old system (or staging environment) to the newly deployed production instance. This phase requires careful coordination with network teams, DNS administrators, and application stakeholders to ensure a smooth transition with minimal user impact. The cutover window should be scheduled during low-traffic periods (typically 02:00-04:00 local time) unless business requirements dictate otherwise.

7.1 DNS Cutover Procedure

DNS cutover involves updating DNS records to point production domains (soc.djezzy.dz, api-soc.djezzy.dz) to the new load balancer IPs. If using a global DNS provider with geo-based routing (such as Route53 or Cloudflare), update records in a controlled manner starting with low-TTL (60 seconds) changes that can be quickly reverted if issues arise. Monitor DNS propagation using multiple resolver locations to confirm global visibility of updated records before announcing the cutover to users.

```
# Update DNS A record (example using Route53 CLI)
aws route53 change-resource-record-sets \
  --hosted-zone-id Z3ABCDEFGHIJKLM \
  --change-batch file://dns-cutover.json

# Verify DNS propagation
dig soc.djezzy.dz @8.8.8.8 +short
dig soc.djezzy.dz @1.1.1.1 +short

# Monitor TTL countdown
watch -n 5 dig soc.djezzy.dz +noall +answer
```

7.2 Traffic Verification

Following DNS propagation, verify that user traffic is reaching the new production infrastructure by monitoring access logs, request metrics, and active session counts. Confirm that SSL/TLS termination is functioning correctly and that clients are not receiving certificate warnings. Validate that authentication flows complete successfully and that existing user sessions (if migrated) remain valid. Enable verbose logging temporarily to capture any client-side errors that may indicate compatibility issues with the new deployment.

7.3 Stakeholder Communication

Once traffic verification confirms successful cutover, initiate the stakeholder communication plan. Send notifications to all user groups announcing the production launch, including links to documentation, support contacts, and known issues or feature differences from the previous system. Schedule kickoff calls with key stakeholder groups (SOC analysts, management, compliance officers) to demonstrate new capabilities and gather initial feedback. Update status dashboards and internal wikis to reflect the production state.

8. Phase 6: Hypercare Support (T+3 to T+30 days)

The hypercare period provides enhanced monitoring and rapid response capability during the critical first 30 days following production launch. During this period, the deployment team maintains elevated availability for incident response, conducts daily health reviews, and implements quick-turn fixes for any issues discovered under real production load. Hypercare ends when the system demonstrates stable operation for 30 consecutive days with no severity-1 or severity-2 incidents.

8.1 Enhanced Monitoring

During hypercare, reduce all alert thresholds to 50% of normal values to detect anomalies earlier and provide more time for investigation before user impact occurs. Enable additional logging verbosity for troubleshooting purposes, particularly around authentication flows, database queries, and external integration calls. Create dedicated dashboards for hypercare-specific metrics including deployment- related errors, new user signups, feature adoption rates, and performance regression detection. Conduct twice-daily reviews of all dashboards with the full deployment team present.

8.2 Incident Response Protocol

Establish a dedicated hypercare war room (virtual or physical) with continuous coverage during business hours and on-call rotation for after-hours incidents. Define escalated SLAs for hypercare period: Severity-1 (service down) targets 5-minute response and 30-minute resolution; Severity-2 (degraded) targets 15-minute response and 2-hour resolution; Severity-3 (minor) targets 1-hour response and 24-hour resolution. All incidents must be documented with post-mortem follow-up required for any severity-1 or severity-2 event regardless of resolution time.

8.3 Daily Health Review Agenda

Time	Topic	Owner	Duration
09:00	Overnight incident review	On-call Lead	15 min
09:15	Dashboard metric walkthrough	SRE Team	20 min
09:35	User feedback and tickets review	Support Lead	15 min
09:50	Capacity and performance trends	Platform Team	15 min
10:05	Action items and blockers	All	10 min

9. Rollback Procedures

Rollback procedures provide a safety net for reverting to a known-good state if the deployment encounters critical issues that cannot be quickly resolved. Three tiers of rollback are defined, each with increasing scope and recovery time objective. The decision to execute rollback rests with the deployment lead in consultation with the product owner, considering factors such as user impact, data loss risk, and estimated time to fix forward versus rollback.

9.1 Tier 1: Application Rollback (RTO: 5 minutes)

Tier 1 rollback reverts the application to the previous Helm release version while preserving the current infrastructure and database state. This is the fastest rollback option and should be the first choice for application-level issues such as crashing pods, failed health checks, or obvious code defects. Execute `helm rollback soc-platform 1` to revert to the prior release version, then verify pod health and application functionality. Document the reason for rollback and preserve logs for post-mortem analysis.

```
# Check available rollback versions
helm history soc-platform -n cybersoc

# Rollback to previous version
helm rollback soc-platform 1 -n cybersoc --wait

# Verify rollback success
helm status soc-platform -n cybersoc
kubectl rollout status deployment/soc-platform-api -n cybersoc
```

9.2 Tier 2: Database Rollback (RTO: 30 minutes)

Tier 2 rollback addresses data corruption or migration failure scenarios by restoring the PostgreSQL database from the pre-migration backup taken at the start of Phase 2. This procedure requires application downtime while the database is restored and consistency is verified. Before executing database rollback, confirm that the issue cannot be resolved with targeted data fixes, as full database restoration is a significant operation with potential for data loss if transactions occurred after the backup was taken.

9.3 Tier 3: Full Environment Rollback (RTO: 2 hours)

Tier 3 rollback is the nuclear option that reverts the entire environment to its pre-deployment state, including infrastructure changes, application deployment, and database state. This tier is reserved for catastrophic failures affecting multiple system components simultaneously or when the root cause cannot be identified within acceptable downtime thresholds. Full environment rollback requires coordination across all teams and should be treated as a last resort after exhausting all other options.

10. Emergency Contacts & Escalation

Maintain an up-to-date contact list for all personnel involved in the deployment and hypercare periods. Contacts should include multiple communication channels (mobile phone, Slack DM, email) and clearly define primary and secondary contacts for each role. Review and update this list before each deployment cycle and confirm availability with all listed personnel during the pre-deployment briefing.

Role	Primary Contact	Secondary	Escalation
Deployment Lead	SOC Platform Lead	DevOps Manager	CTO
Platform/SRE	Senior SRE Engineer	Platform Architect	VP Engineering
Database (DBA)	Senior DBA	DBA Team Lead	Director of Data
Security	Security Engineer	CISO Office	Chief Security Officer
Network/Infra	Network Engineer	Infrastructure Lead	VP Infrastructure
Product Owner	SOC Product Manager	Director of Product	CEO
Executive Sponsor	VP of Engineering	CTO	CEO

10.1 External Vendor Contacts

Vendor	Service	Contact Method	Contract Number
Cloud Provider	Infrastructure Support	Premium Support Portal	CP-2024-SOC-001
DNS Provider	DNS Management	Enterprise Support Line	DNS-2024-PROD-042
Certificate Authority	TLS Certificates	cert-manager Support	N/A (Let's Encrypt)
Security Scanner	Penetration Testing	Account Team Direct	SEC-2024-SCAN-015